



The Observer Pattern

Human Computer Interaction Research
University of Nevada, Reno



Design Pattern Collection

□ Behavioral Patterns

- » Strategy

- » Observer

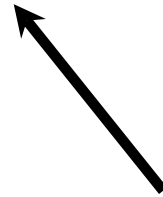
problem

Related objects need to
communicate with each
other to maintain
consistency

example: eye of Sauron



Frodo is coming!!!



Sauron



Saruman

Initial attempt

```
public class EyeOfSauron {
```

```
    public void enemiesDetected() {
```

```
        int elves = getNumberOfElves();
```

```
        int hobbits = getNumberOfHobbits();
```

```
        int dwarves = getNumberOfDwarves();
```

```
        sauronDisplay.update(elves,hobbits,dwarves);
```

```
        sarumanDisplay.update(elves,hobbits,dwarves);
```

```
    }
```

```
// other methods
```

```
}
```

coded to
concrete
implementation

what if we want
to add this
guy???



tasty
hobbits!!!

Hur

Publisher Subscriber

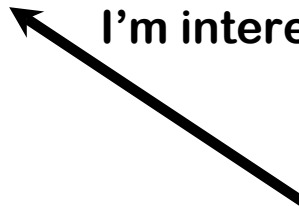
I'll keep an eye out



I'm interested



I'm interested



Publisher Subscriber

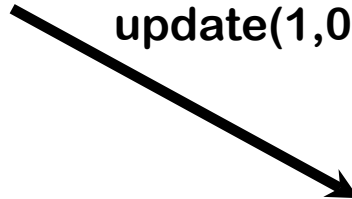
a dwarf and an Elf!!!



update(1,0,1)



update(1,0,1)



Publisher Subscriber

boohooo ;-(



unsubscribe!!



Publisher Subscriber

I'll keep an eye out



I better get
some help



subscribe!!



Publisher Subscriber

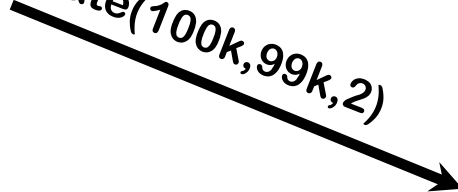
uh oh!!!



update(100k,99k,2)



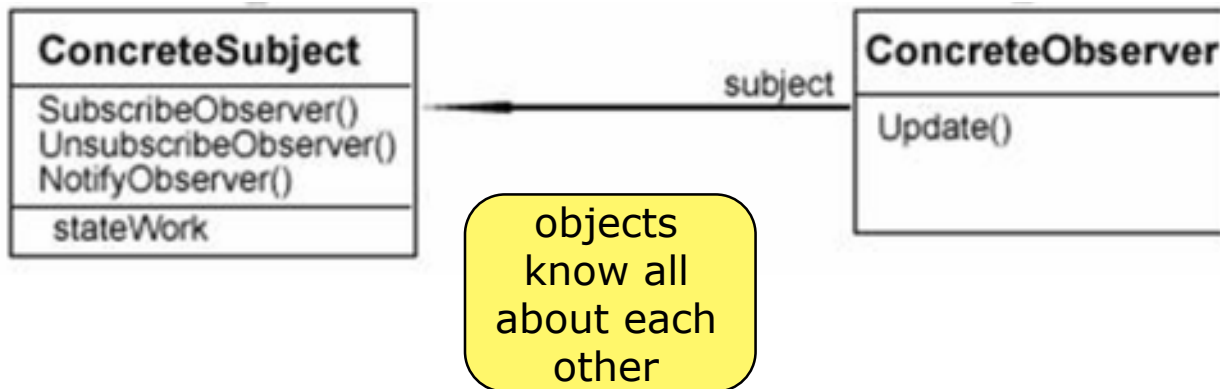
update(100k,99k,2)



Solution

The Observer Pattern defines a one to many dependency between objects so that when one object changes state, all of its dependents are notified and updated automatically.

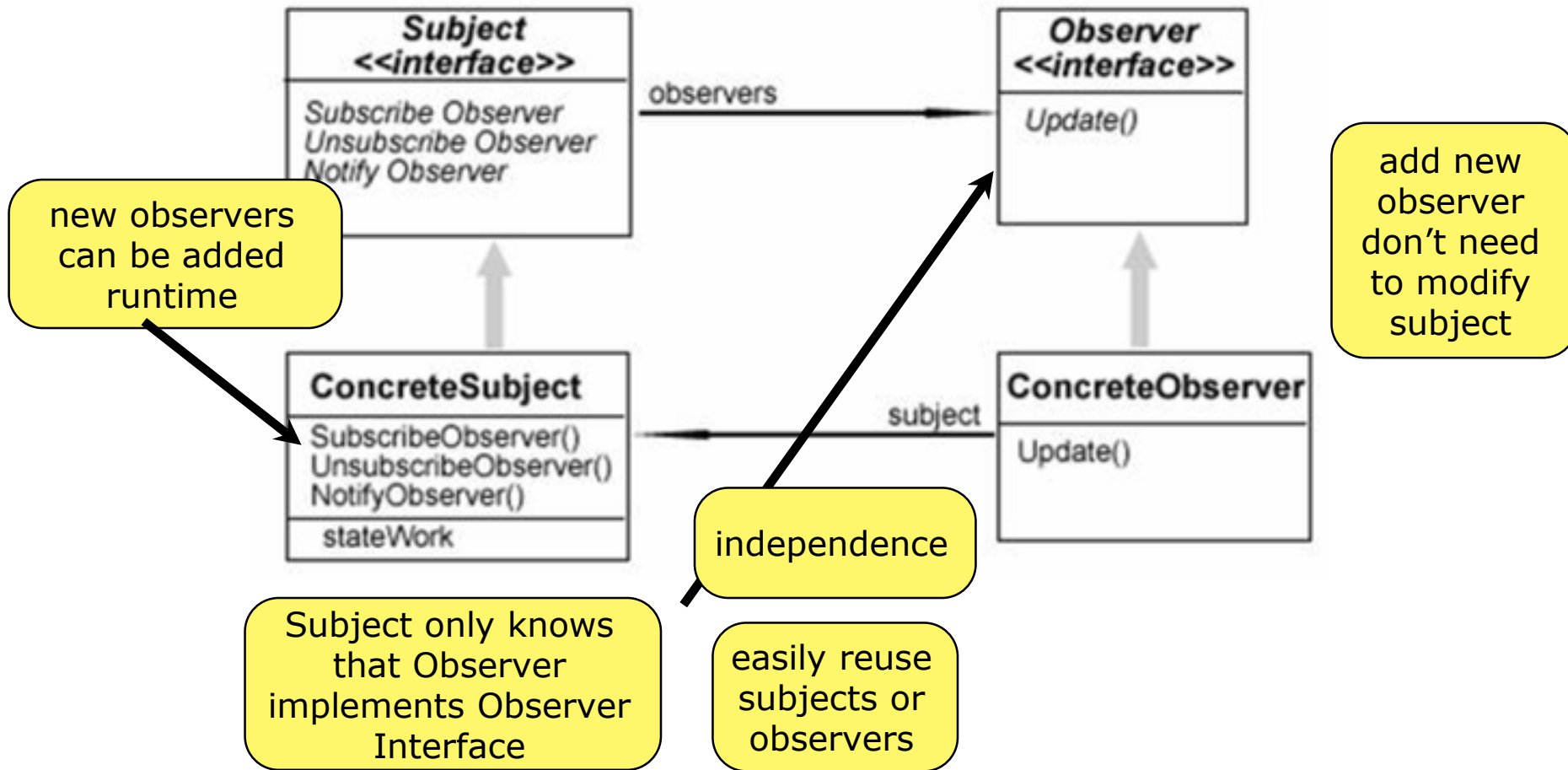
How it was



Design Principle

Strive for Loosely coupled designs between objects that interact

Class Diagram



Interfaces

```
public interface Subject {  
    public void registerObserver(Observer o);  
    public void removeObserver(Observer o);  
    public void notifyObservers();  
}
```

```
public interface Observer {  
    public void update(int elves, int dwarves, int hobbits);  
}
```

Specific
information

Concrete Subject

```
public class EyeOfSauron implements Subject{

    private ArrayList observers;
    private int elves;
    private int hobbits;
    private int dwarves;

    public EyeOfSauron(){
        observers = new ArrayList();
    }

    public void enemiesDetected() {
        notifyObservers();
    }

    public void notifyObservers() {
        for (int i=0; i< observers.size(); i++) {
            Observer observer = (Observer) observers.get(i);
            observer.update(elves,hobbits,dwarves);
        }
    }
}
```

Concrete Subject

```
public void setEnemies(int elves, int dwarves, int hobbits) {  
    this.elves=elves;  
    this.dwarves=dwarves;  
    this.hobbits=hobbits;  
    enemiesDetected();  
}
```

```
public void registerObserver(Observer o) {  
    observers.add(o);  
}
```

```
public void removeObserver(Observer o) {  
    int i = observers.indexOf(o);  
    if (i >= 0) {  
        observers.remove(i);  
    }  
}
```

Concrete Observer

```
public class SarumanDisplay implements Observer {  
  
    private int elves;  
    private int hobbits;  
    private int dwarves;  
    private String name;  
    private Subject eyeData;  
  
    public SarumanDisplay (Subject eye, String name) {  
        this.eyeData=eye;  
        this.name=name;  
        eyeData.registerObserver(this);  
    }  
  
    public void update(int elves, int dwarves, int hobbits) {  
        this.elves=elves;  
        this.dwarves=dwarves;  
        this.hobbits=hobbits;  
        System.out.println(name+ " Display update");  
        display();  
    }  
}
```

Concrete Observer

```
public void display(){
    System.out.println(name + " sees:" + elves+" elves and " + dwarves + "
dwarves and " + hobbits + " hobbits");
}

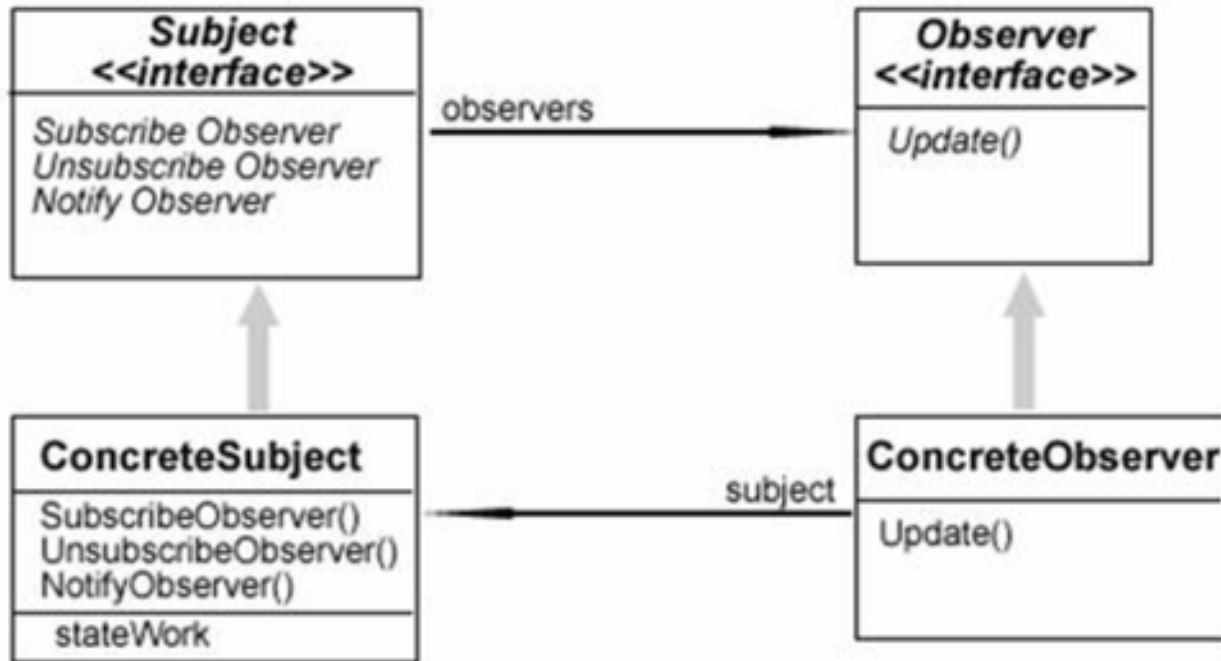
public void loseInterest() {
    eyeData.removeObserver(this);
    System.out.println("remove Observer");
}
```

Test App.

```
public class TestSauronEye {  
  
    public static void main(String[] args) {  
        EyeOfSauron eye = new EyeOfSauron();  
  
        SarumanDisplay sarumanDisplay = new SarumanDisplay(eye, "Saruman Display");  
        SauronDisplay sauronDisplay = new SauronDisplay(eye, "Sauron Display");  
  
        eye.setEnemies(1, 1, 2);  
  
        sauronDisplay.loseInterest();  
  
        eye.setEnemies(4, 2, 2);  
    }  
}
```

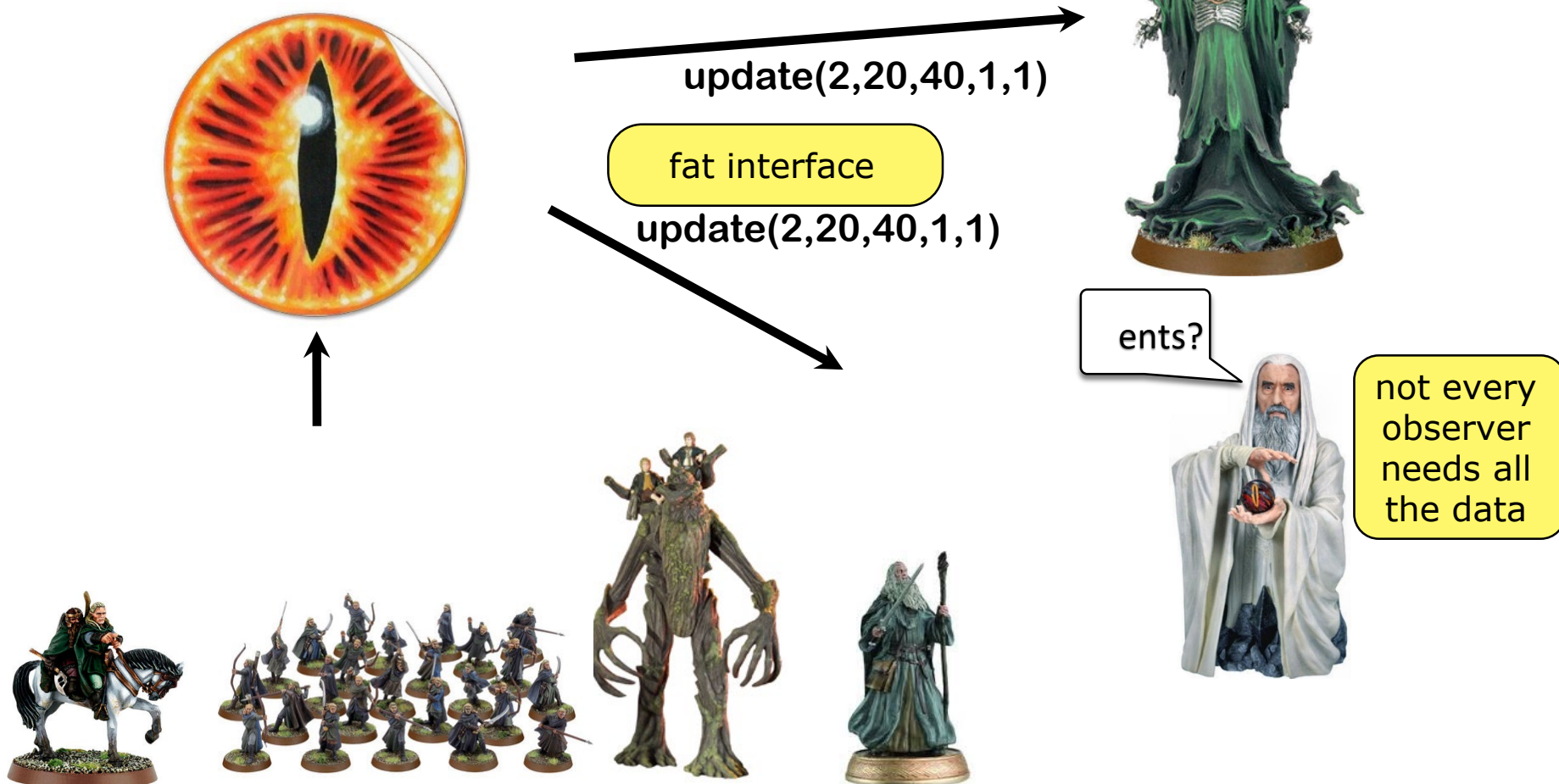
```
bin $java TestSauronEye  
SarumanDisplay sees: 1 elves and 2 dwarves and 1 hobbit  
SauronDisplay sees: 1 elves and 2 dwarves and 1 hobbit  
Remove Observer  
SarumanDisplay sees: 4 elves and 2 dwarves and 2 hobbit
```

Observer Class Diagram



disadvantages of Push

2 hobbits, 20 dwarves, 40 elves, 1 ent, 1 wizard



Push vs Pull

- All the data gets pushed all the time

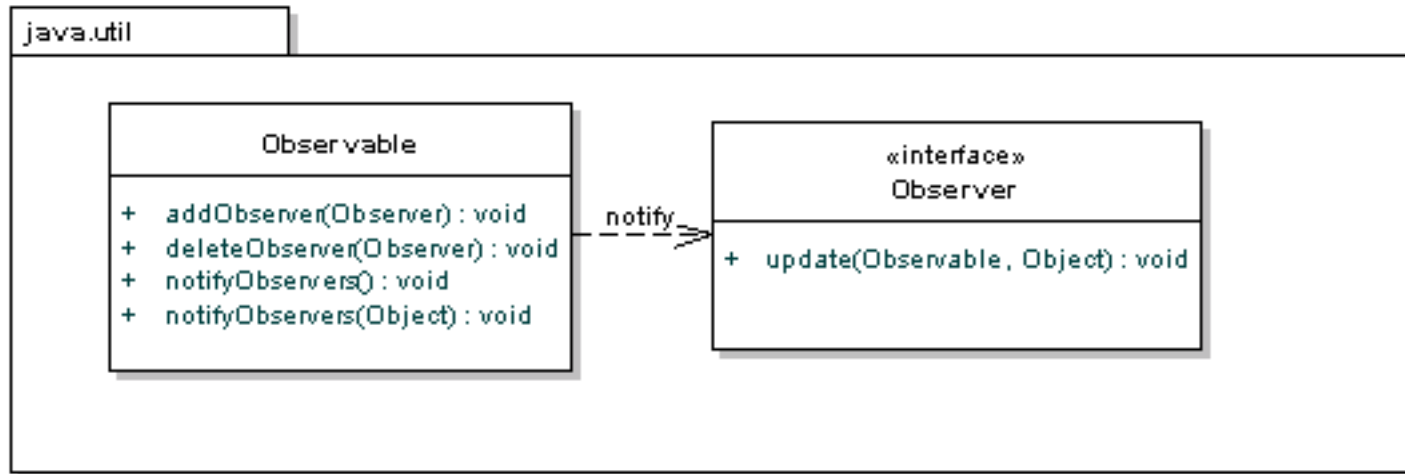
- + every observer has all the latest data

- May need multiple calls to get all the data

- + more flexibility
let observers figure it out themselves

Java Observable

this is an abstract class



Java does all the
work with notifying
observers

Observable == Subject

projector

desk

Group 1

Group 2

Group 3

Group 4

Group 5

Group 6

Group 7

Group 8

Group 9

Group 10

Group 11

Group 12

pillar

Group 13

Group 14

Group 15

Group 16

Group 17

Group 18

Group 19

Group 20

Group 21

Group 22

Group 23

Group 24

Group 25

Group 26

Group 27

Group 28

Group 29

pillar

skateboards

door